

*Software per la gestione di una biblioteca  
universitaria*

---

**Identificativo progetto:** G8

**Gruppo numero:** 8

**Nome documento:** Progetto Ingegneria del software

**Data documento:** 8/12/2025

**Team:** Alessandro Picariello, Lorenzo Trovato, Matteo Iandiorio, Daniele Manzo

# 1 Introduzione

## 1.1 Obiettivi

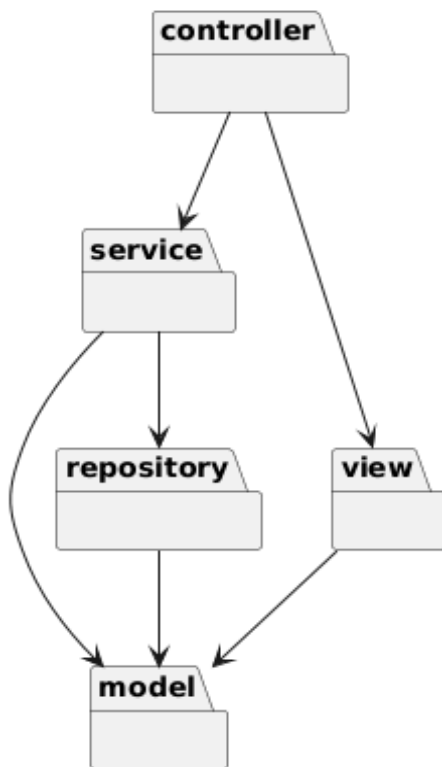
Il presente documento si prefigge l'obiettivo di esplicitare le scelte di design effettuate, nonché di fornire la descrizione dei modelli statici e dei modelli dinamici e dell'architettura del sistema nella sua totalità.

## 1.2 Architettura del sistema

Il sistema si divide in 5 moduli chiave:

- 1) **Model**: il *model* fornisce i metodi per accedere ai dati utili all'applicazione
  - Componenti: InterfaceID, Dati, Libro, Utente e Prestito.
- 2) **View**: la *view* visualizza i dati contenuti nel model e si occupa dell'interazione con utenti e agenti
  - Componenti: InterfaceView<T>, BaseView<T>, LibroView, UtenteView, PrestitoView.
- 3) **Controller**: il *controller* riceve i comandi dell'utente (in genere attraverso la view) e li attua modificando lo stato degli altri due componenti
  - Componenti: InterfaceController, BaseController<T>, CrudController<T>, LibroController, UtenteController, PrestitoController.
- 4) **Repository**: il repository gestisce l'accesso e la memorizzazione dei dati, isolando al resto del sistema i dettagli dell'IO (Input/Output)
  - Componenti: InterfaceGestoreIO<T> e GestoreFile<T>.
- 5) **Service**: il service contiene le regole e la logica di business del sistema
  - Componenti: InterfaceService<T>, BaseService<T>, LibroService, UtenteService, PrestitoService.

## Diagramma dei package



- **service -> repository** : Il service si occupa di salvare, modificare, cercare ed eliminare gli oggetti.
- **controller -> service** : Il controller si occupa di gestire e coordinare le richieste dell'utente, tuttavia non contiene la logica di business.
- **controller -> view** : il controller gestisce la view, come avviene nel modello MVC.
- **view -> model** : la view ha bisogno di accedere ai dati, di cui il model è a conoscenza. L'interazione diretta serve anche ad evitare ulteriori dipendenze con il repository o con il service.
- **repository -> model** : il repository deve salvare e/o caricare i dati del model.
- **service -> model** : il service deve applicare le logiche di business sul model.

Pattern principali usati:

Model-View-Controller MVC  
Repository

Principi di buona progettazione impiegati:

DRY

KISS

SINE

YAGNI

SoC

Principio della minima sorpresa

Regola del boy-scout

Principi di buona progettazione orientata agli oggetti impiegati:

Principio della singola responsabilità

Principio di segregazione delle interfacce

Principio di sostituzione di Liskov

Principio aperto/chiuso

Principio di inversione della dipendenza

## **Viste architetturelle impiegate:**

**Vista logica:** definizione dei moduli costituenti e delle relazioni che intercorrono tra essi, mediante l'uso del diagramma delle classi

**Vista di processo:** definizione delle interazioni tra i componenti del sistema, a tempo di esecuzione.

## **Stili architetturelle adottati:**

**Model-View-Controller Design Pattern:** stile architetturelle che prevede la separazione del software in tre componenti: Model (adibito alla gestione dei dati), View (preposto alla visualizzazione all'utente) e Controller (designato al coordinamento).

**Repository Design Pattern:** stile architetturelle che consente di astrarre l'accesso e la gestione dei dati, separandoli dalla logica applicativa del software.

## **Approcci alla scomposizione impiegati:**

**Approccio funzionale o procedurale:** Suddivisione delle funzionalità da realizzare in operazioni più semplici.

**Approccio object-oriented:** Suddivisione in classi, tesa alla descrizione del dominio del problema.

## Principi generali di buona progettazione impiegati:

**DRY (Don't Repeat Yourself) e KISS (Keep It Simple, Stupid!):** L'uso del polimorfismo e dei generics consente di ridurre la ridondanza nel codice, favorendo la flessibilità e la compattezza.

Esempi all'interno del codice:

È stata evitata la scrittura dei metodi **salva**, **cerca**, **elimina** per ognuna delle classi del model (Libri, Utenti, Prestiti), favorendo una singola implementazione all'interno delle classi generiche **Repository<T>** e **BaseService<T>**.

**SINE (Simple Is Not Easy):** È stato effettuato un processo di adattamento del codice, finalizzato al conseguimento di una maggiore semplicità di utilizzo. Tuttavia, il trade-off è rappresentato dalla crescita della complessità del codice, dovuto precipuamente all'impiego dei generics.

**Principio della minima sorpresa:** È stata adoperata la convenzione di denominazione standard di Java, a cui si è affiancata un'accurata scelta nell'attribuzione di identificatori tesi a riflettere le funzionalità del codice. Inoltre, è stata ridotta al minimo la ridondanza.

**YAGNI (You Aren't Going to Need It):** È stata effettuata l'eliminazione di diversi metodi, al fine di evitare l'implementazione di funzionalità superflue.

**Regola del boy-scout:** Nelle diverse fasi di manutenzione del codice è stato effettuato un graduale affinamento della qualità del codice, con particolare riguardo per il miglioramento della chiarezza del codice.

## Principi di buona progettazione orientata agli oggetti:

**Principio della singola responsabilità:** È stata operata una suddivisione delle classi, in maniera tale che ciascuna di esse effettui un compito ben preciso.

**Principio di segregazione delle interfacce:** L'inserimento di un'interfaccia in ogni macroarea è finalizzato a ridurre la presenza di lunghe catene di dipendenze.

- Il **BaseService** non dipende dalla classe concreta **Repository**, ma dall'interfaccia **IRepository<T>**.
- Il **Repository** non dipende dalla classe concreta **GestoreFile**, ma dall'interfaccia **IGestoreIO<T>**.

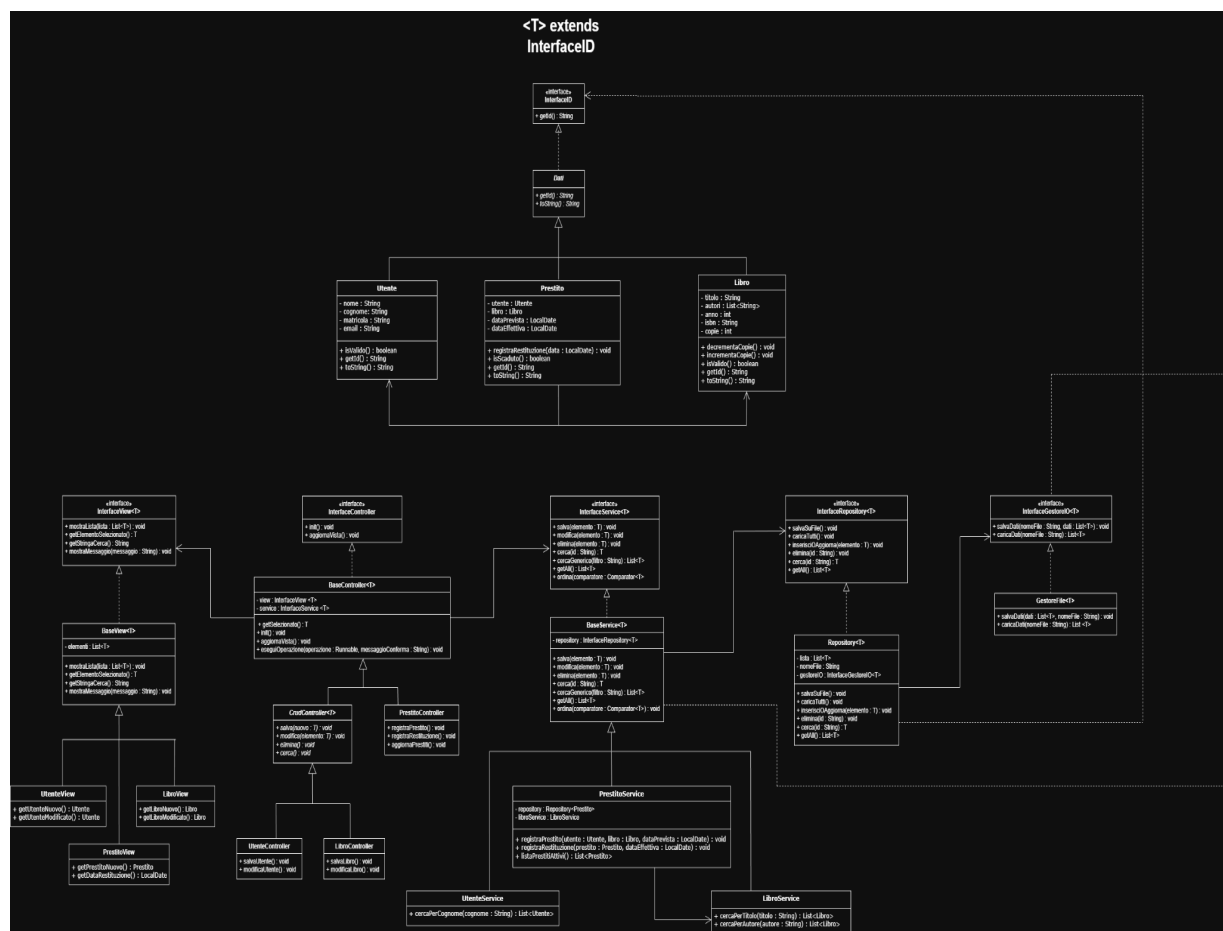
**Principio di sostituzione di Liskov:** Le classi astratte sono state implementate in modo tale che gli oggetti possano essere sostituiti con istanze dei loro sottotipi, senza determinare alterazioni alla correttezza del programma

**Principio aperto-chiuso:** L'overriding dei metodi della superclasse all'interno delle sottoclassi e l'incapsulamento consentono di apportare modifiche alle classi di base, evitando che si riflettano sulle classi derivate.

## Separation of Concerns (SoC - Separazione degli Interessi)

È stata effettuata una separazione dell'applicazione in cinque livelli: **View** , **Controller** , **Service**, **Repository**, **Model**.

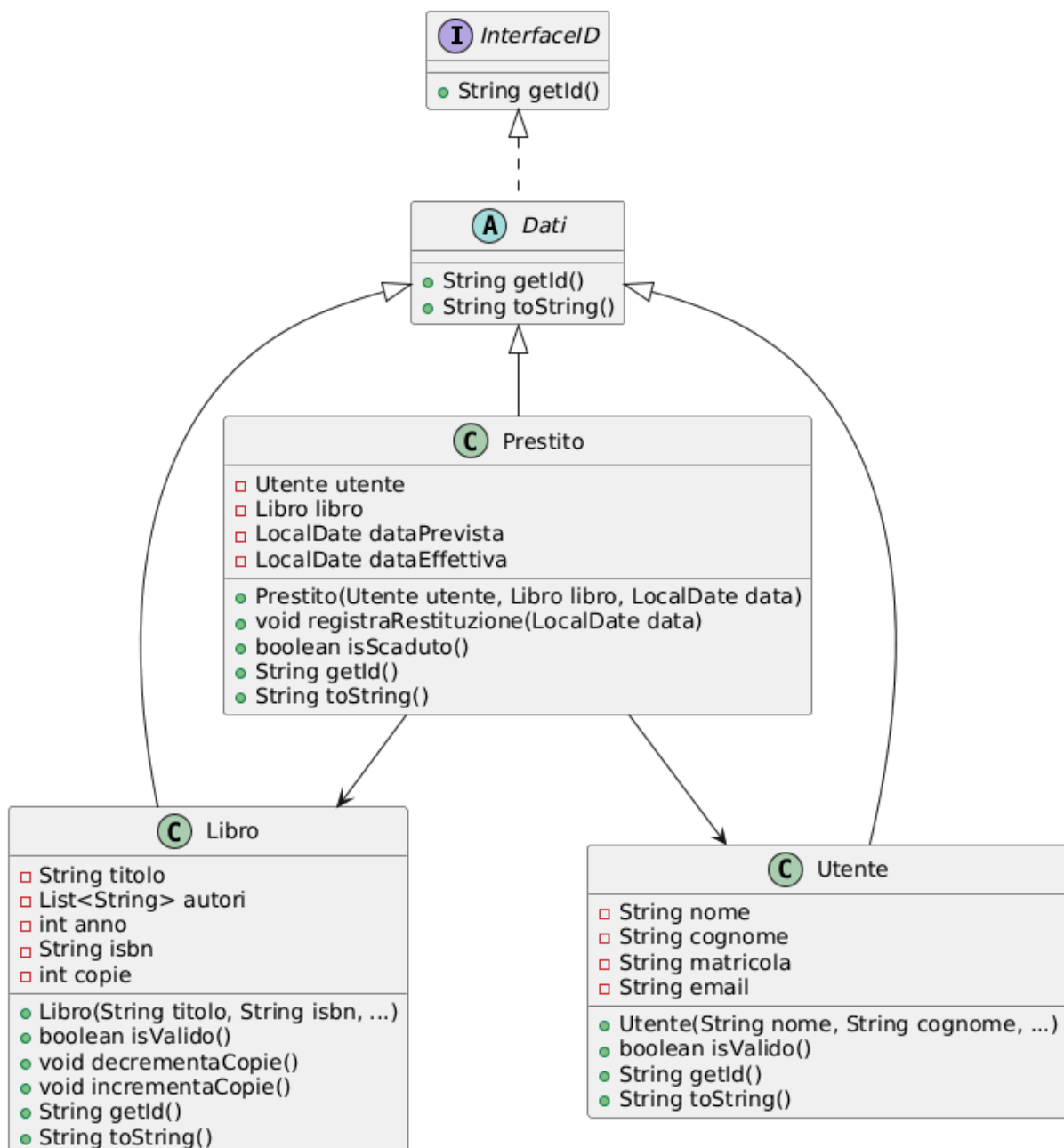
## 2 Diagramma delle classi



## 2.1 Struttura

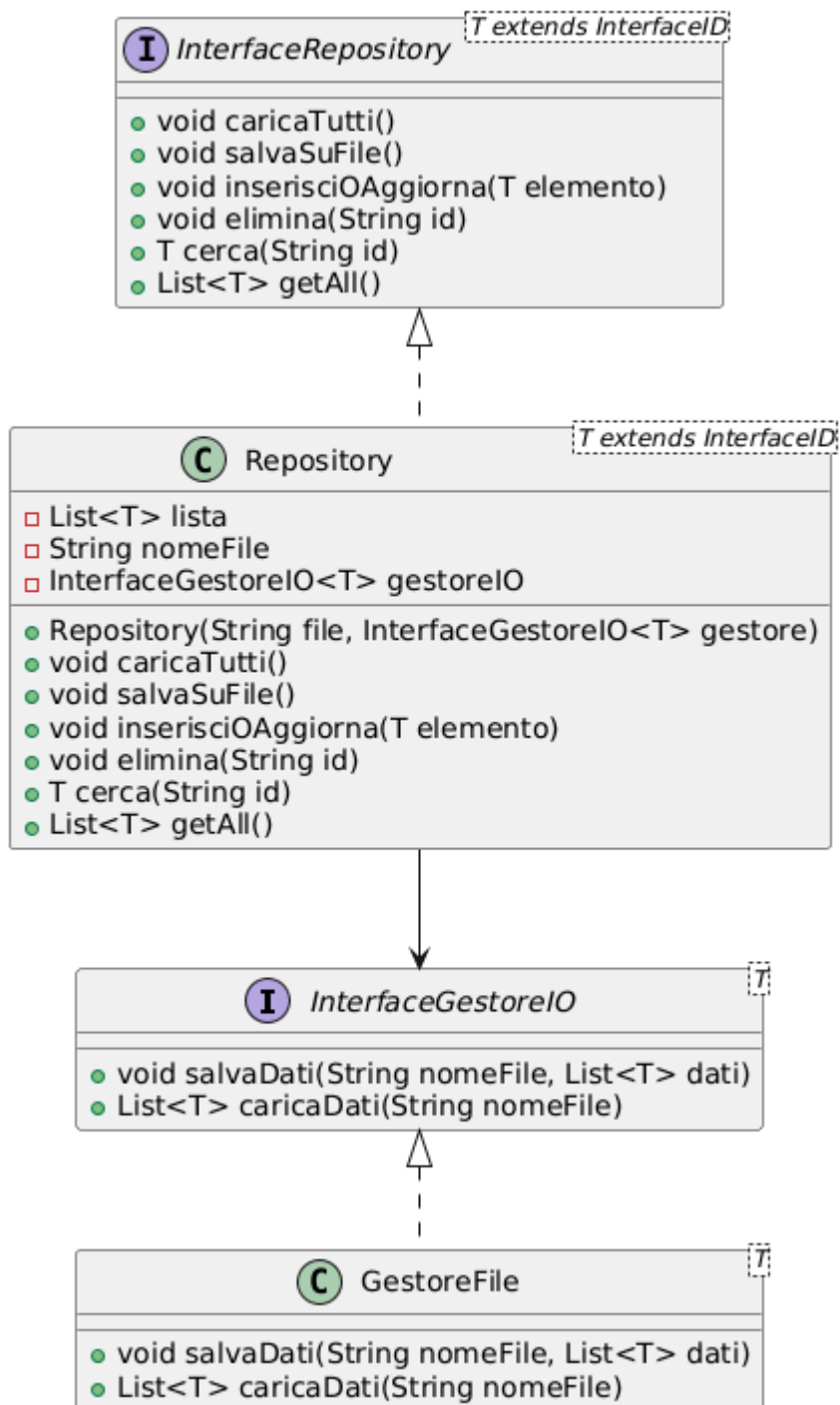
### Model

Il modulo Model si occupa di rappresentare le entità con le quali opera il sistema.



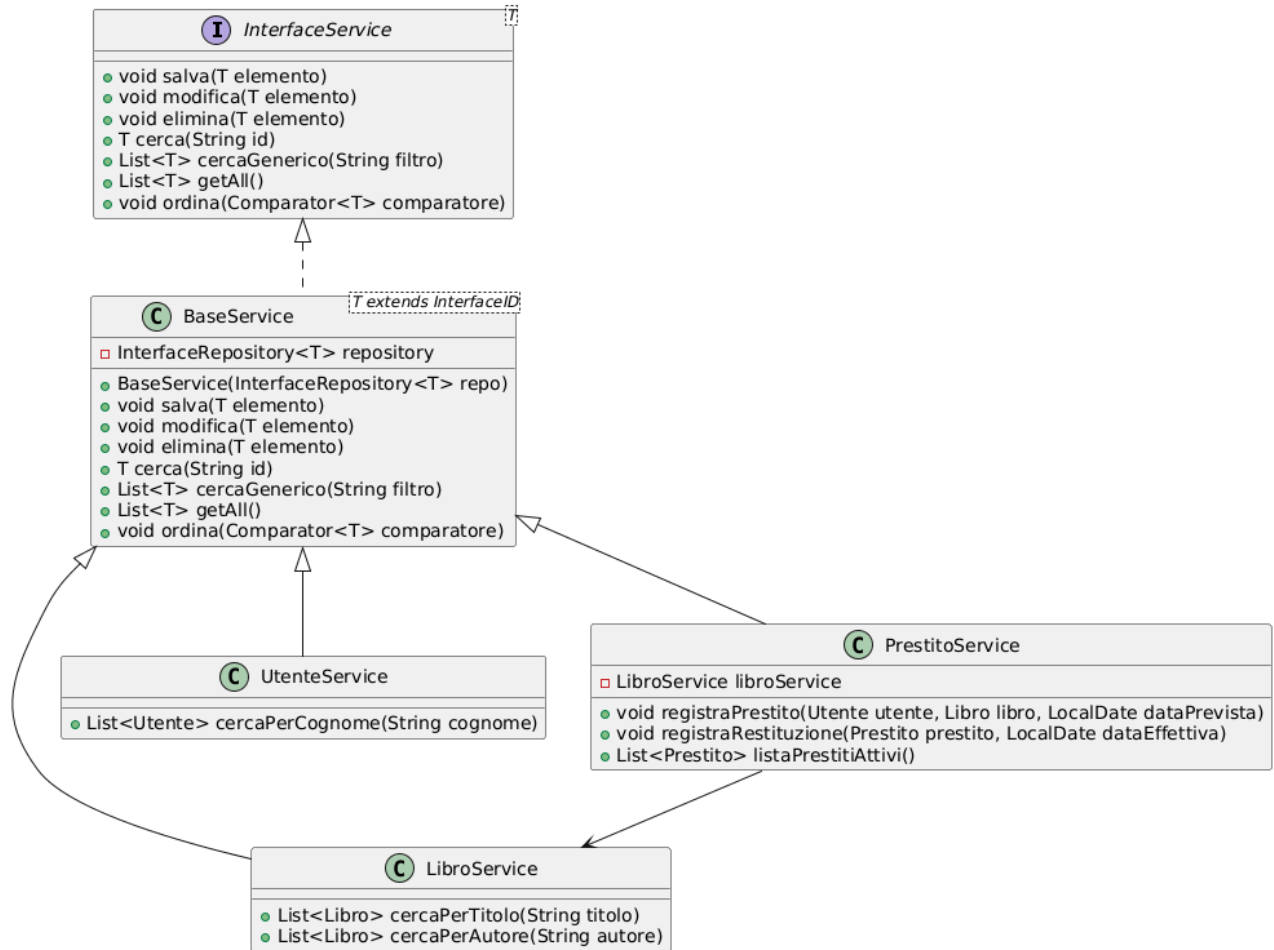
### Repository

Il modulo Repository è deputato all'accesso e al salvataggio dei dati, garantendone la persistenza.



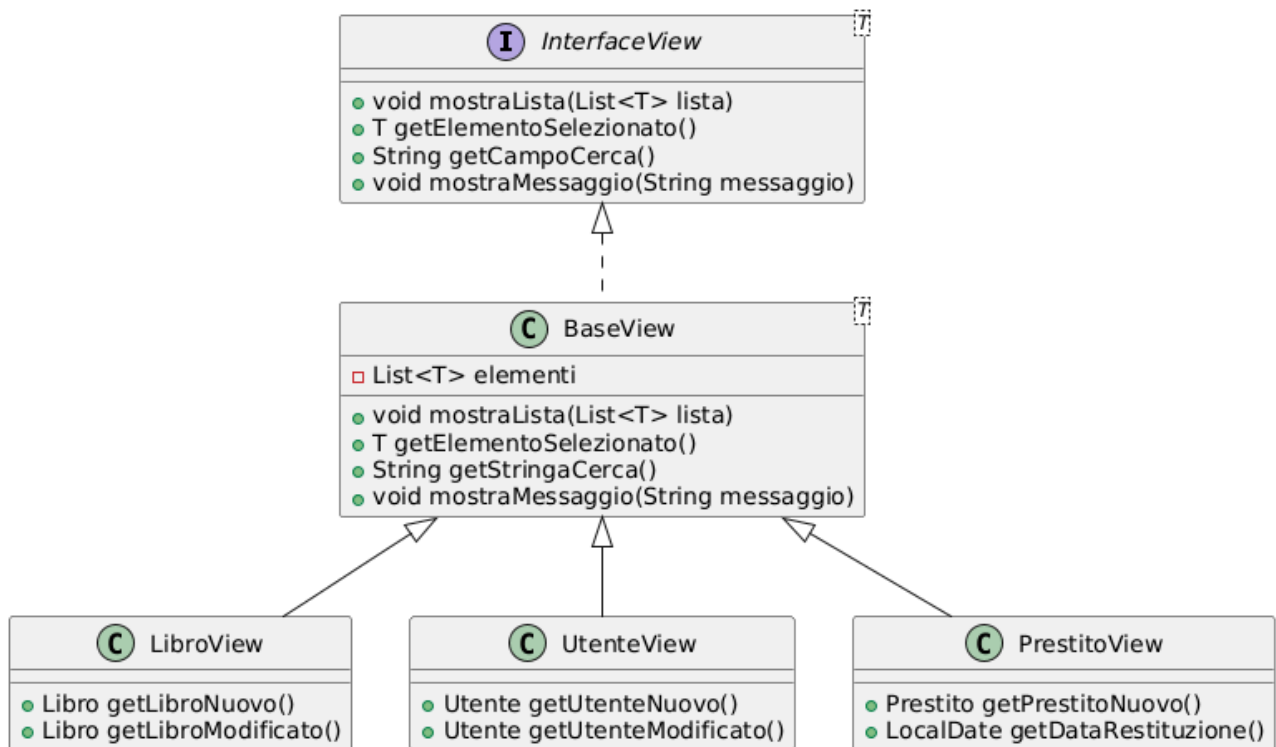
## Service

Il modulo Service si occupa dell'implementazione delle effettive funzionalità messe a disposizione dal sistema, nonché dell'orchestrazione dell'applicazione stessa. Opera sui dati forniti dal Repository.



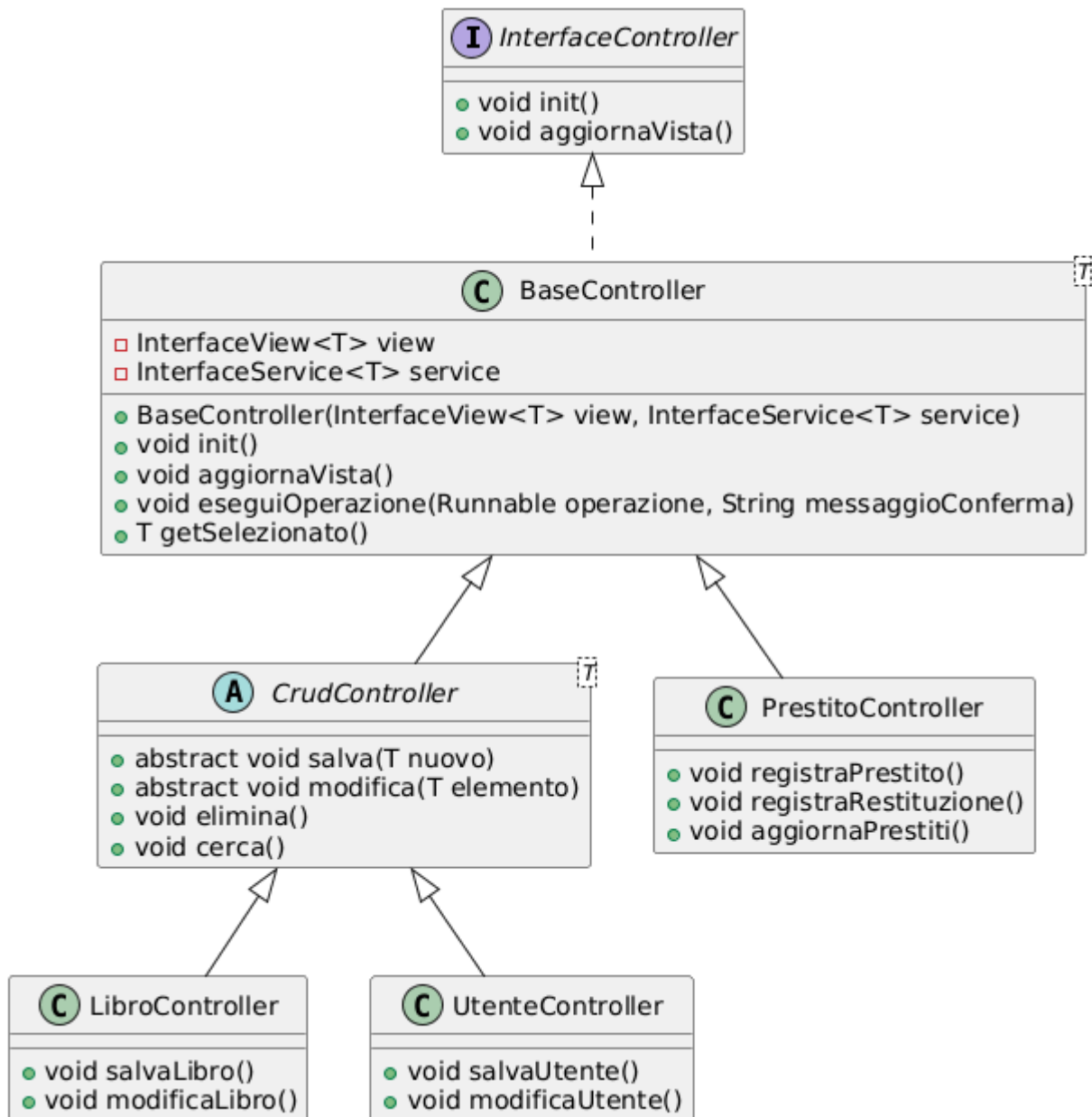
## View

Il modulo View è responsabile dell'interfaccia utente, della visualizzazione dei dati e della raccolta dell'input inserito dal gestore del sistema.



## Controller

Il modulo Controller si occupa di gestire e coordinare il flusso dell'applicazione, fungendo da intermediario tra la View ed il Service. Deve rispondere agli eventi generati in risposta alle azioni intraprese dal gestore del sistema, invocare i metodi del Service e aggiornare la View.

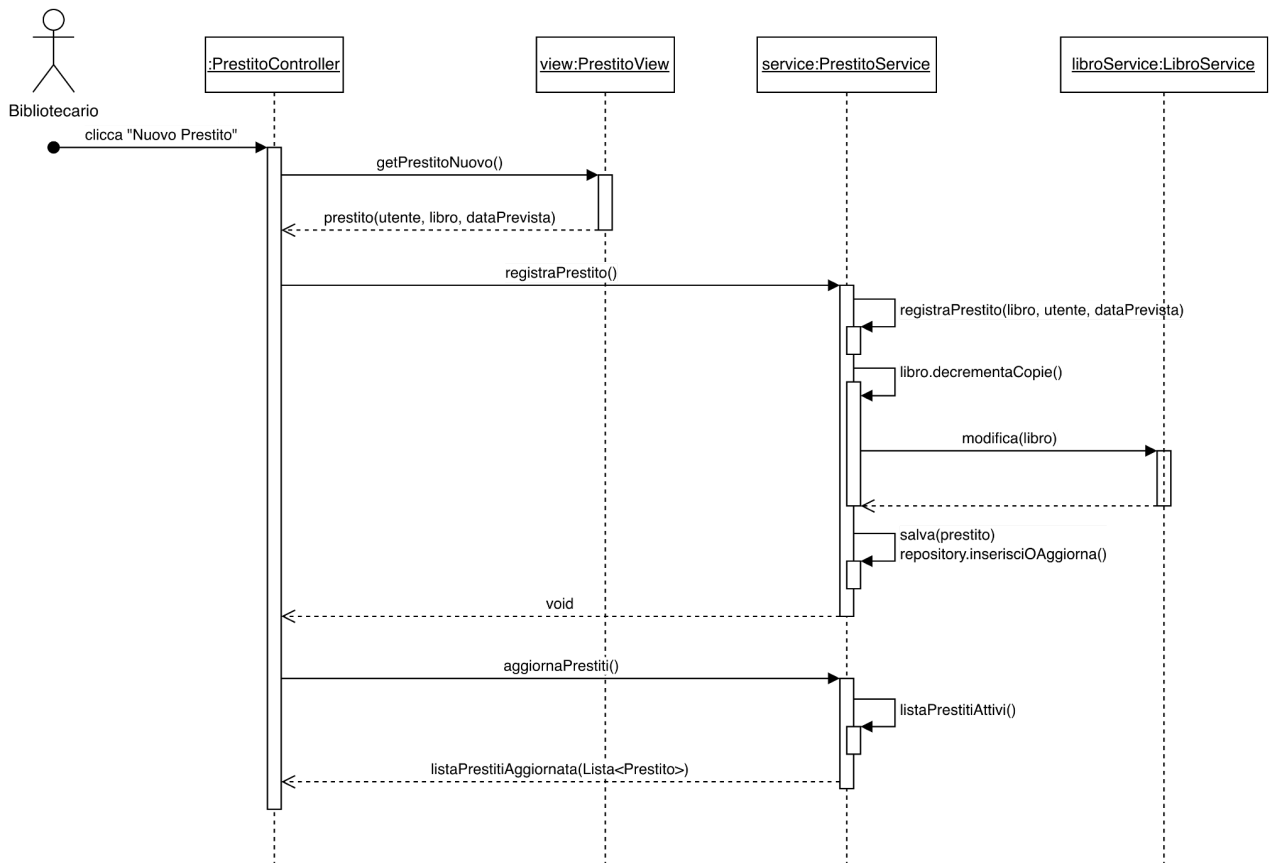


### 3. Modello dinamico

Diagramma di sequenza

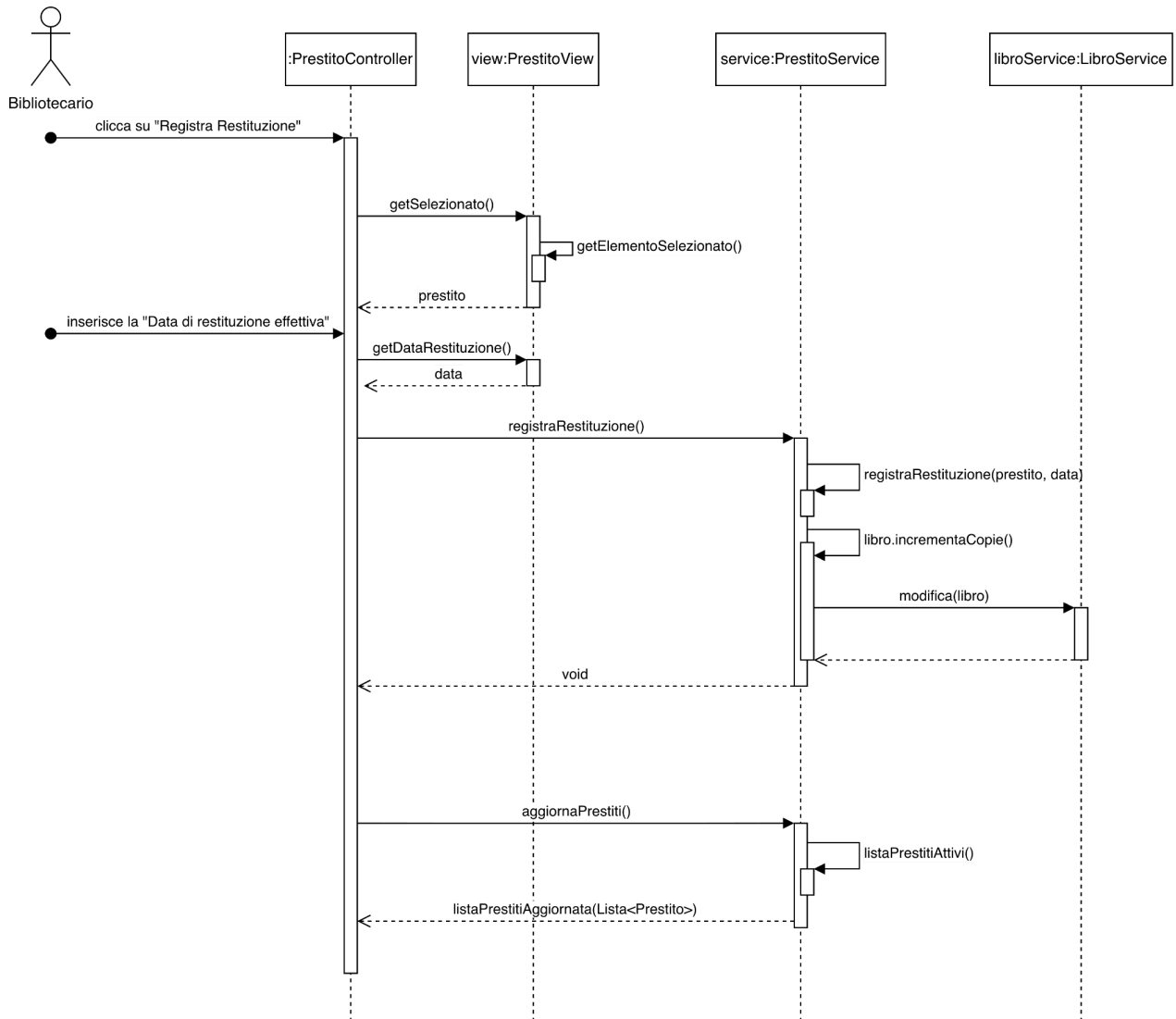
## Prestito

Registrazione prestito



**Diagramma raffigurante la fase di registrazione del prestito:**

1. Il **:PrestitoController** raccoglie i dati del prestito dalla View.
2. Il Controller effettua la chiamata del metodo **:registraPrestito()** su **PrestitoService**.
3. Il **:PrestitoService** registra il prestito ed esegue una transazione a due fasi:
4. Chiama **libro.decrementaCopie()** e delega l'aggiornamento del libro al **:LibroService**.
5. Salva il nuovo oggetto **prestito**.
6. Il Controller aggiorna la lista dei prestiti (**aggiornaPrestiti()**) richiamando la lista dei prestiti attivi..



### Diagramma raffigurante la fase di registrazione della restituzione:

1. Il **:PrestitoController** raccoglie i dati del prestito dalla View.
2. Il Controller effettua la chiamata del metodo **getSelezionato()** su **PrestitoView**, che restituisce l'elemento selezionato; successivamente, chiama **getDataRestituzione()** e restituisce la data.
3. Il **:PrestitoService** registra la restituzione ed esegue una transazione a due fasi:
  - Invoca **libro.incrementaCopie()** e delega l'aggiornamento del libro al **:LibroService**.
  - Effettua il salvataggio del **prestito**.
4. Il **:PrestitoController** richiede l'aggiornamento della lista dei prestiti, invocando il metodo **aggiornaPrestiti()**. Viene restituita la lista dei prestiti attivi, provvista delle modifiche apportate.

# Libro

## Cerca Libro id

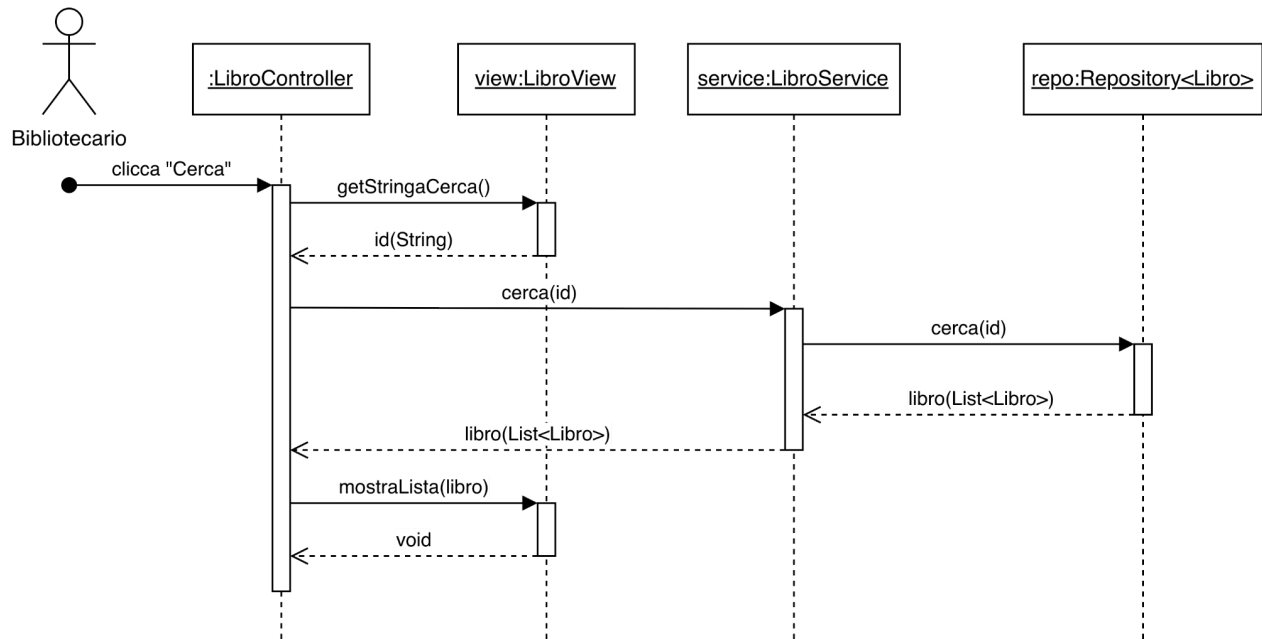
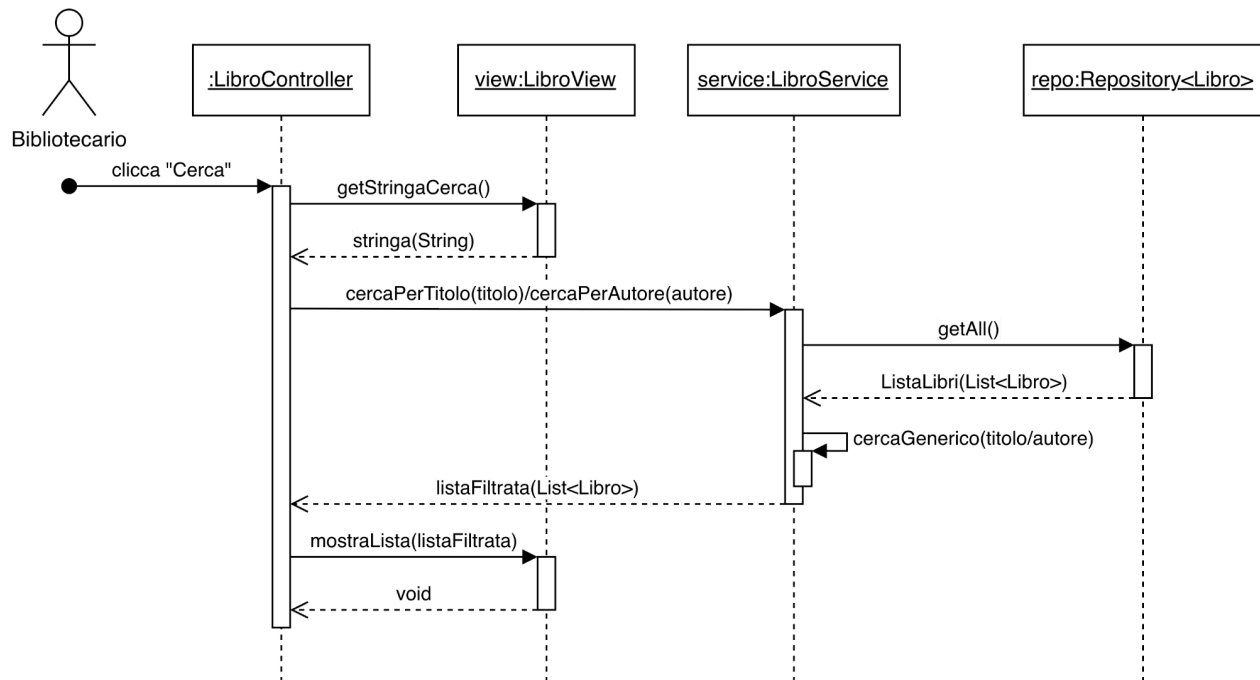


Diagramma raffigurante la fase di ricerca di un libro tramite id:

1. Il **:LibroController** ottiene l'id del libro dalla View.
2. Chiama **cerca(id)** sul **:LibroService**.
3. Il Service chiama **cerca(id)** sul **repo:Repository**.
4. Il Repository restituisce il libro corrispondente.
5. La View mostra la lista dei libri contenente quello desiderato.

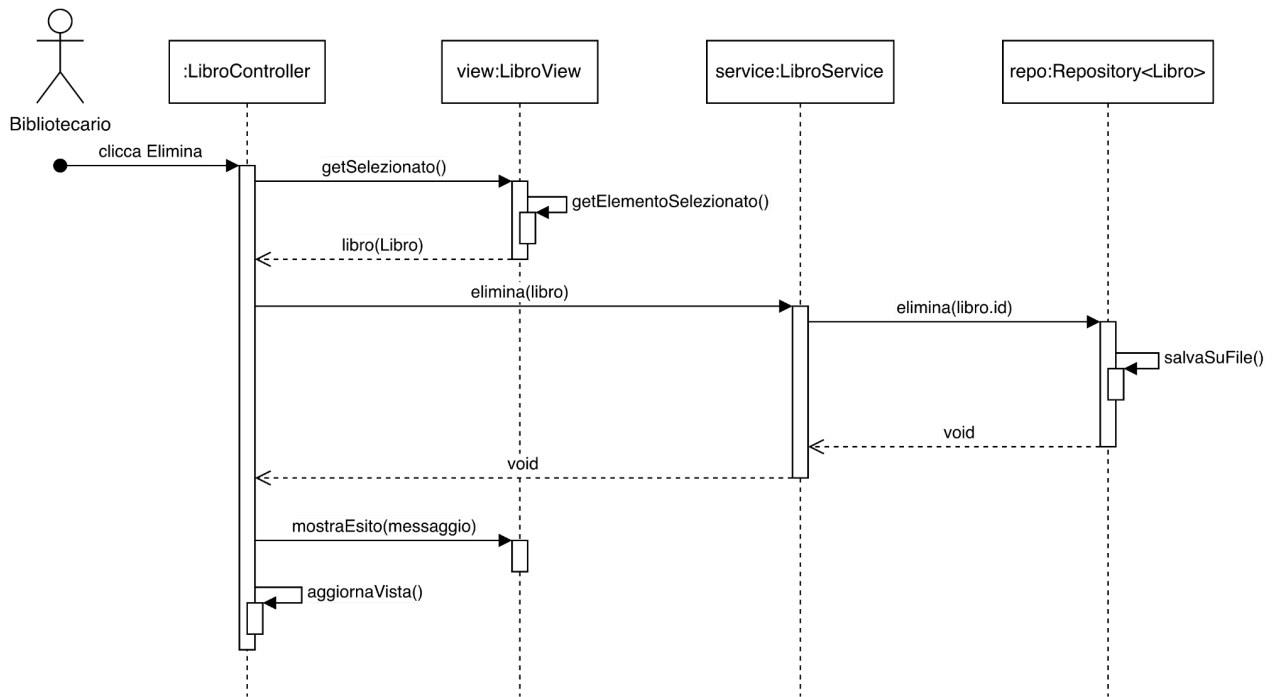
**cerca libro filtrato**



#### Diagramma raffigurante la fase di ricerca filtrata di un libro:

1. Il **:LibroController** cerca la Stringa del libro dalla View.
2. Chiama **cercaPerTitolo(titolo)/cercaPerAutore(autore)** sul **:LibroService**.
3. Il Service chiama **getAll()** sul **repo:Repository**.
4. Il Repository restituisce la lista dei libri corrispondenti, successivamente Service cerca la stringa tra i generics e restituisce la lista filtrata..
5. Il Controller chiama **mostraLista(listaFiltrata)** e View mostra la lista dei libri contenente la lista filtrata.

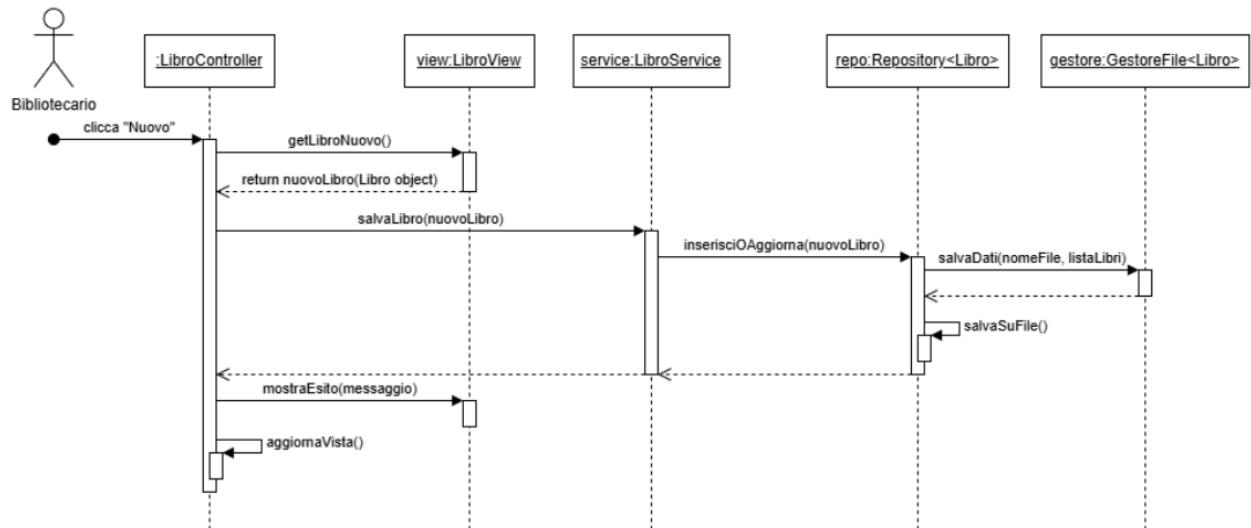
#### Elimina libro



**Diagramma raffigurante la fase di eliminazione di un libro:**

1. Il **:LibroController** ottiene il libro da eliminare dalla View.
2. Chiama **elimina(libro)** sul **:LibroService**.
3. Il Service chiama **elimina(libro.id)** sul **repo:Repository**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la rimozione.
5. Il Controller notifica l'esito e aggiorna la vista.

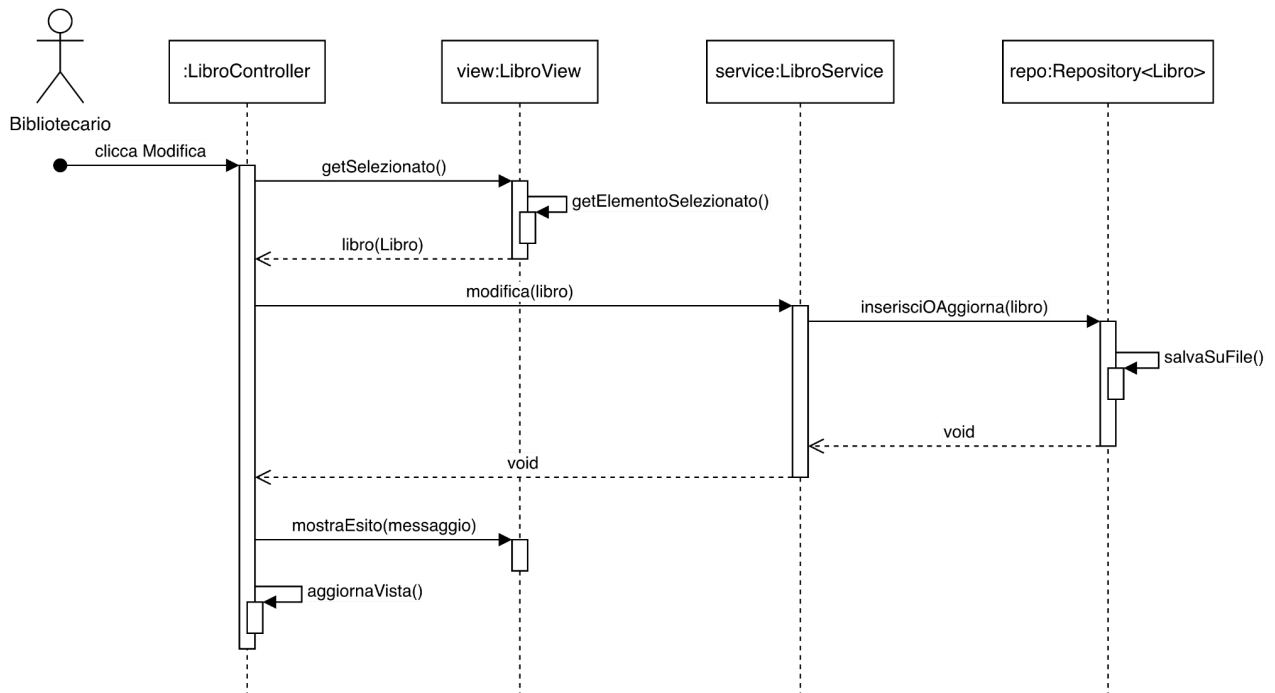
**Inserimento libro**



### Diagramma raffigurante la fase di aggiunta di un nuovo libro:

1. Il **:LibroController** ottiene il libro da aggiungere dalla View.
2. Chiama **salvaLibro(nuovoLibro)** sul **:LibroService**.
3. Il Service chiama **InserisciOAggiorna(nuovoLibro)** sul **repo:Repository**.
4. Il Repository chiama **salvaDati(nomeFile, lista)** sul **gestore:GestoreFile**.
5. Il Repository esegue **salvaSuFile()** per rendere permanente l'aggiunta.
6. Il Controller notifica l'esito e aggiorna la vista.

### Modifica libro

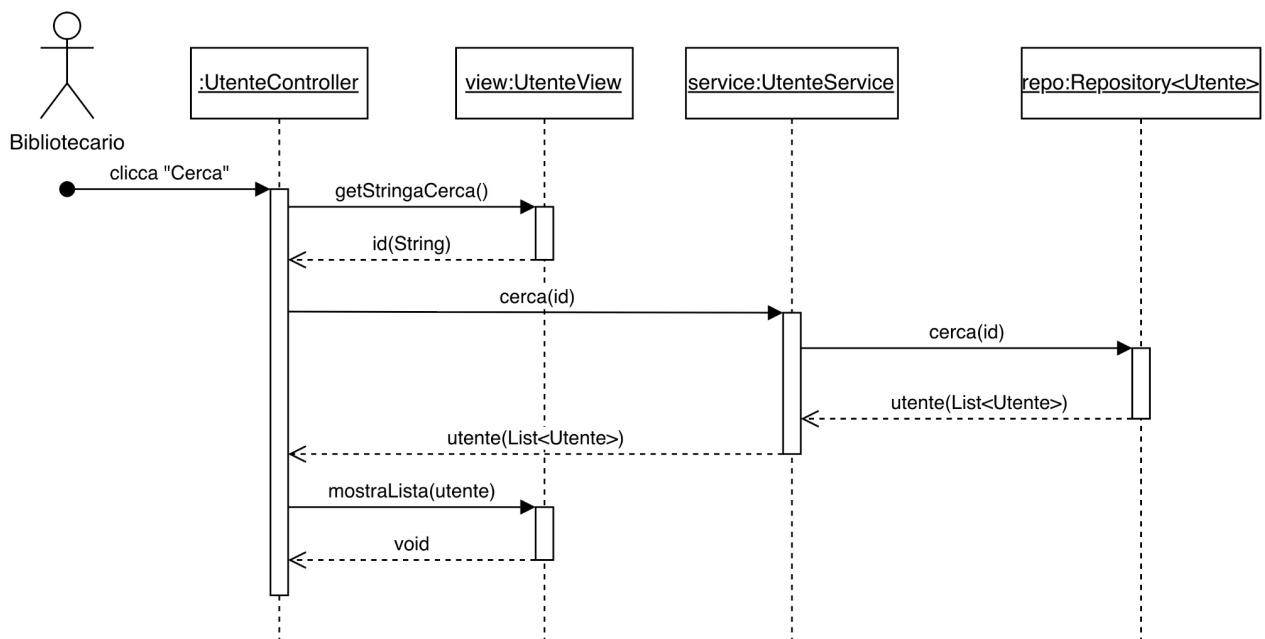


### Diagramma raffigurante la fase di modifica di un libro:

1. Il **:LibroController** ottiene il libro da modificare dalla View.
2. Chiama **modifica(libro)** sul **:LibroService**.
3. Il Service chiama **InserisciOAggiorna(libro)** sul **repo:Repository**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la modifica.
5. Il Controller notifica l'esito e aggiorna la vista.

## Utente

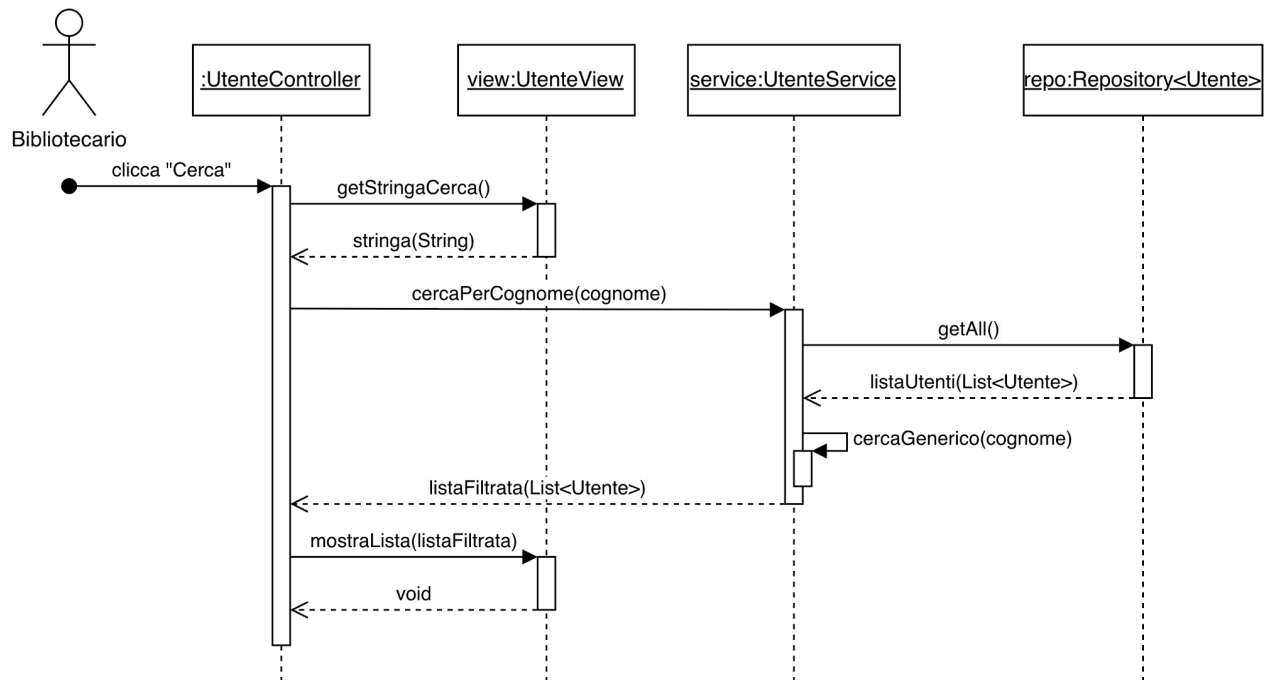
### Cerca Utente Id



### Diagramma raffigurante la fase di ricerca di un utente tramite id:

1. Il **:UtenteController** ottiene l'id del libro dalla View.
2. Chiama **cerca(id)** sul **:UtenteService**.
3. Il Service chiama **cerca(id)** sul **repo:Repository**.
4. Il Repository restituisce l'utente corrispondente.
5. La View mostra la lista degli utente, contenente quello desiderato.

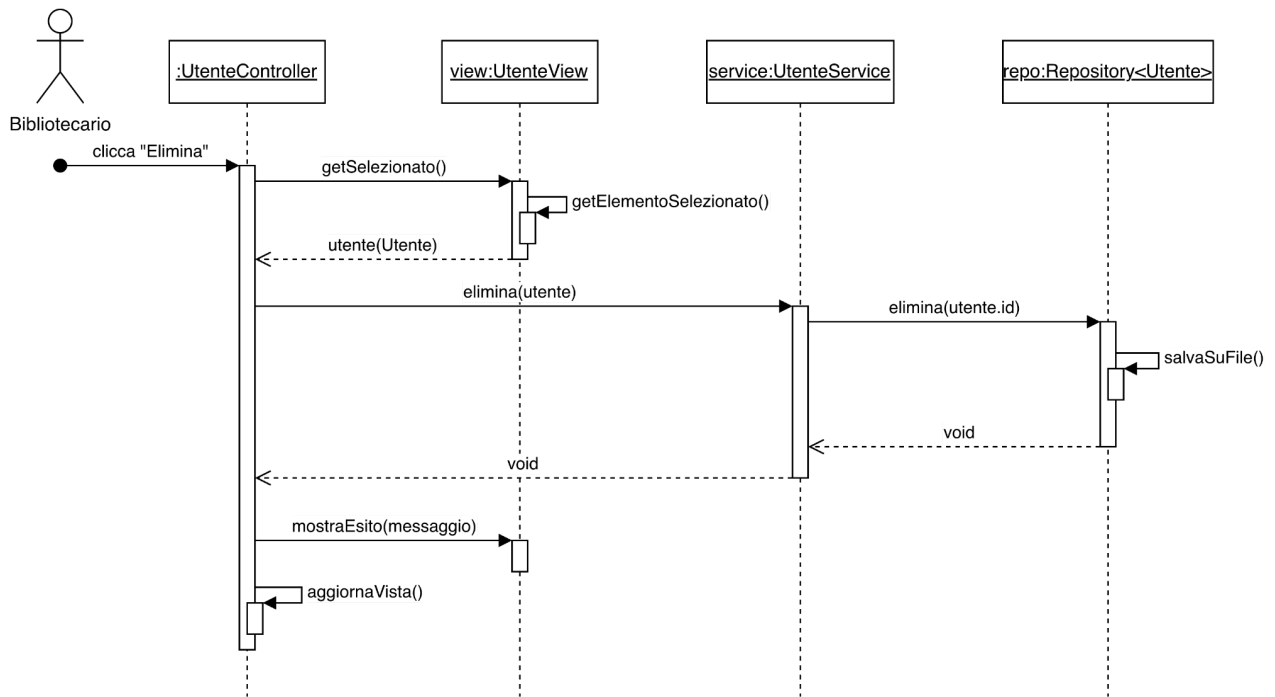
### Cerca utente filtrato



**Diagramma raffigurante la fase di ricerca filtrata di un utente:**

1. Il **:UtenteController** cerca la Stringa relativa all'utente dalla View.
2. Chiama **cercaPerCognome(cognome)** sul **:UtenteService**.
3. Il Service chiama **getAll()** sul **repo:Repository**.
4. Il Repository restituisce la lista degli utenti corrispondenti, successivamente il Service cerca la stringa tra i generics e restituisce la lista filtrata..
5. Il Controller chiama **mostraLista(listaFiltrata)** e View mostra la lista filtrata degli utenti.

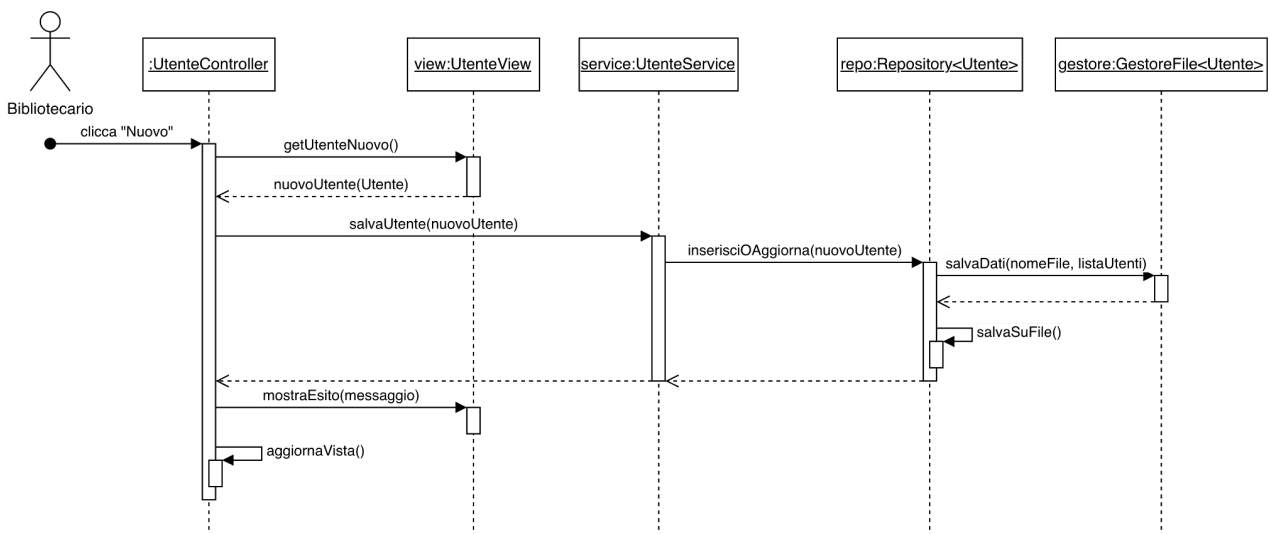
**Elimina utente**



**Diagramma raffigurante la fase di eliminazione di un utente:**

1. Il **:UtenteController** ottiene l'utente da eliminare dalla View.
2. Chiama **elimina(utente)** sul **:UtenteService**.
3. Il Service chiama **elimina(utente.id)** sul **repo:Repository**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la rimozione.
5. Il Controller notifica l'esito e aggiorna la vista.

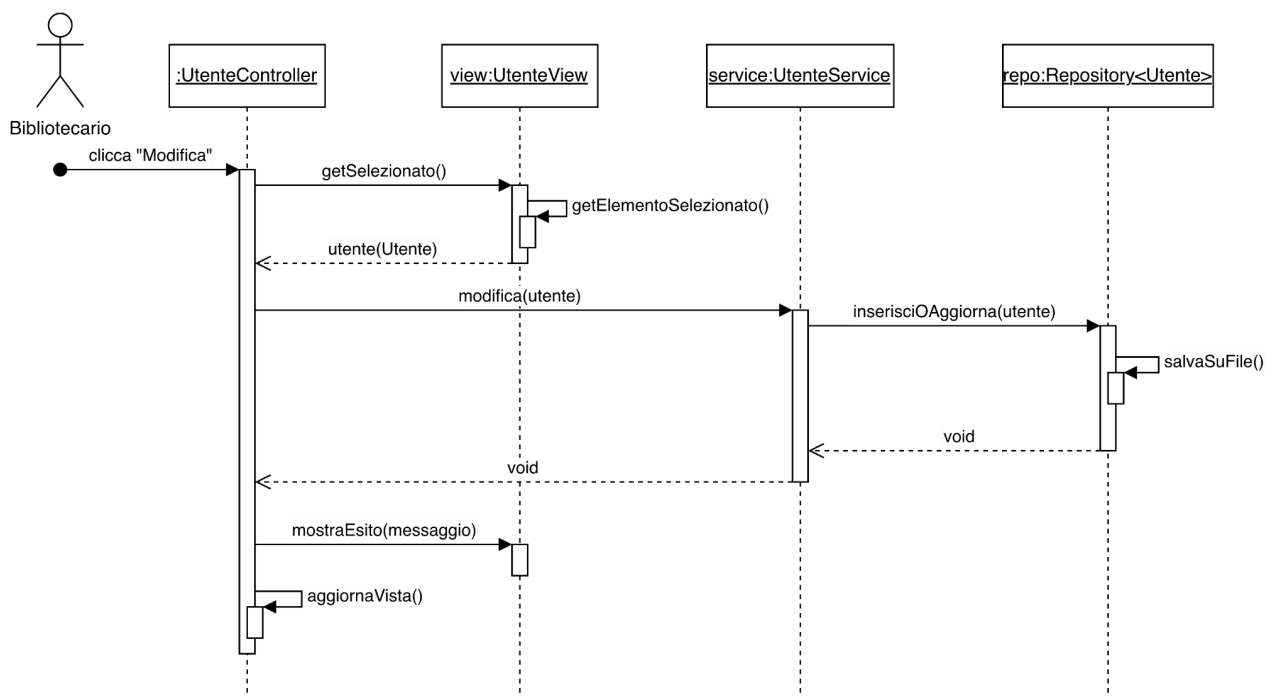
## Inserimento utente



### Diagramma raffigurante la fase di inserimento di un nuovo utente:

1. Il **:UtenteController** ottiene l'utente da aggiungere dalla View.
2. Chiama **salvaUtente(nuovoUtente)** sul **:UtenteService**.
3. Il Service chiama **InserisciOAggiorna(nuovoUtente)** sul **repo:Repository**.
4. Il Repository chiama **salvaDati(nomeFile, listaUtenti)** sul **gestore:GestoreFile**.
5. Il Repository esegue **salvaSuFile()** per rendere permanente l'aggiunta.
6. Il Controller notifica l'esito e aggiorna la vista.

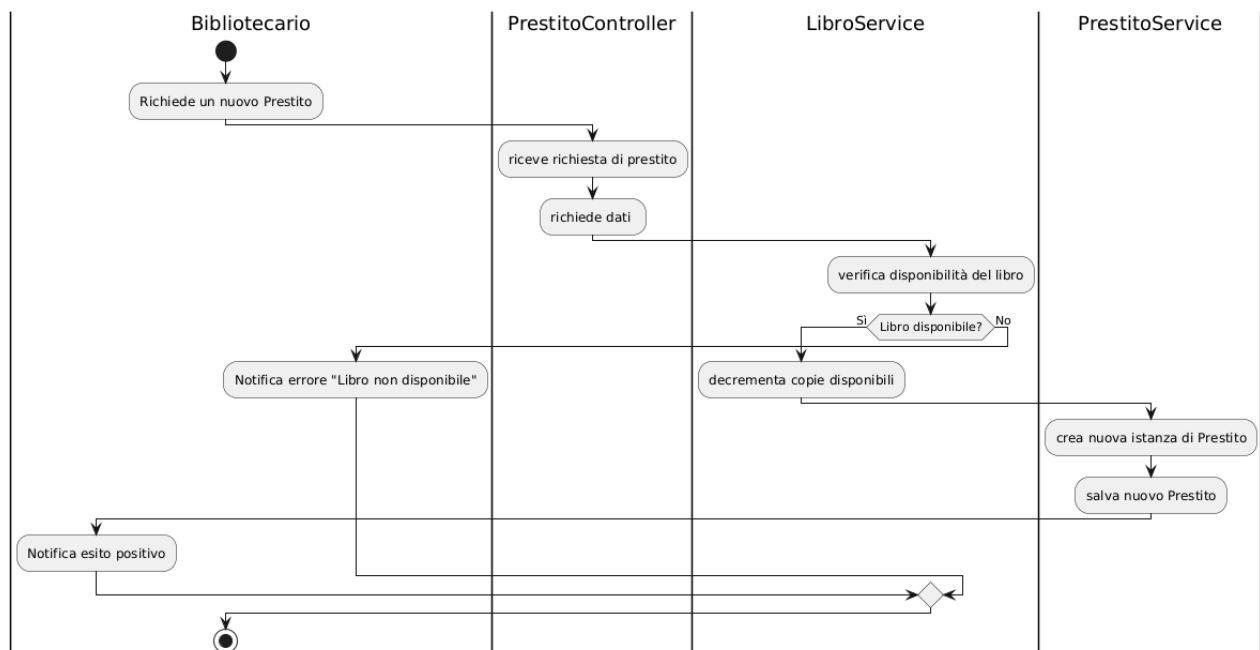
### Modifica utente



### Diagramma raffigurante la fase di modifica di un utente:

1. Il **:UtenteController** ottiene l'utente da modificare dalla View.
2. Chiama **modifica(utente)** sul **:UtenteService**.
3. Il Service chiama **InserisciOAggiorna(utente)** sul **repo:Repository**.
4. Il Repository esegue **salvaSuFile()** per rendere permanente la modifica.
5. Il Controller notifica l'esito e aggiorna la vista.

## Diagramma delle attività



## 1. Inizio e Verifica (PrestitoController)

1. **Richiede un nuovo prestito:** Il Bibliotecario richiede la registrazione di un nuovo prestito.
2. **Riceve richiesta di prestito:** PrestitoController riceve la richiesta di registrazione del prestito.
3. **Richiede dati:** Il Controller necessita dei dati necessari alla registrazione del prestito, quali l'utente richiedente (Utente), il libro di cui si deve effettuare la concessione (Libro), e la data di restituzione (Data Prevista); per tale motivo, **PrestitoController** deve ottenere i dati mediante un'apposita richiesta.
4. **Verifica disponibilità del libro:** Il Controller deve verificare se il libro indicato sia disponibile per il prestito; a tale scopo, delega a LibroService il compito di controllare se siano presenti copie disponibili.

## 2. Decisione Condizionale (Logica di Business)

Il flusso si biforca basandosi sul risultato della verifica di disponibilità:

- **Ramo "No" (Libro non disponibile):**
  1. Se il libro richiesto non è disponibile per il prestito (No), il controllo torna al Bibliotecario.
  2. **Notifica errore "Libro non disponibile":** Viene visualizzato un messaggio volto a notificare un errore.
  3. Unificazione del flusso e terminazione.

- **Ramo "Sì" (Libro disponibile):**

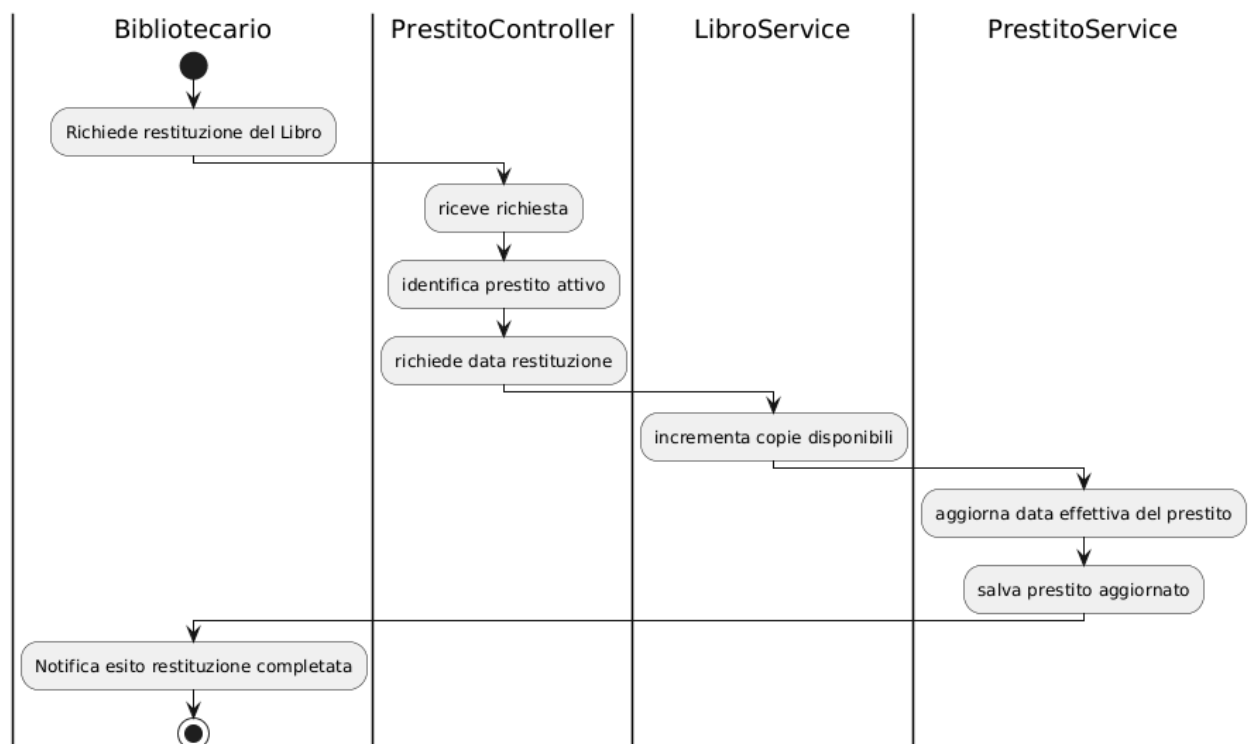
1. Se il Libro è disponibile per il prestito (Sì), LibroService procede.
2. **Decrementa copie disponibili:** LibroService provvede a diminuire il numero di copie disponibili (funzionalità messa a disposizione dalla chiamata del metodo **decrementaCopie()** sul modello del Libro).
3. Il controllo passa a PrestitoService.

### 3. Finalizzazione della Transazione (PrestitoService)

1. **Crea nuova istanza di Prestito:** PrestitoService crea la nuova istanza Prestito con i dati ottenuti.
2. **Salva nuovo Prestito:** Il Service effettua il salvataggio dell'oggetto all'interno del Repository (rendendolo, in tal modo, persistente).
3. **Notifica esito positivo:** Il flusso torna al Bibliotecario; viene notificato l'esito positivo dell'operazione.

### 4. Conclusione

Unificazione e terminazione del flusso.



## 1. Ricezione e Identificazione (PrestitoController)

- Il Bibliotecario avvia il processo, richiedendo la restituzione di un libro.
- PrestitoController riceve la richiesta.
- Il Controller individua il prestito attivo di cui occorre effettuare la restituzione.
- PrestitoController richiede in cui deve avvenire la restituzione, necessaria al calcolo di eventuali ritardi o alla chiusura del prestito.

## 2. Aggiornamento Copie (LibroService)

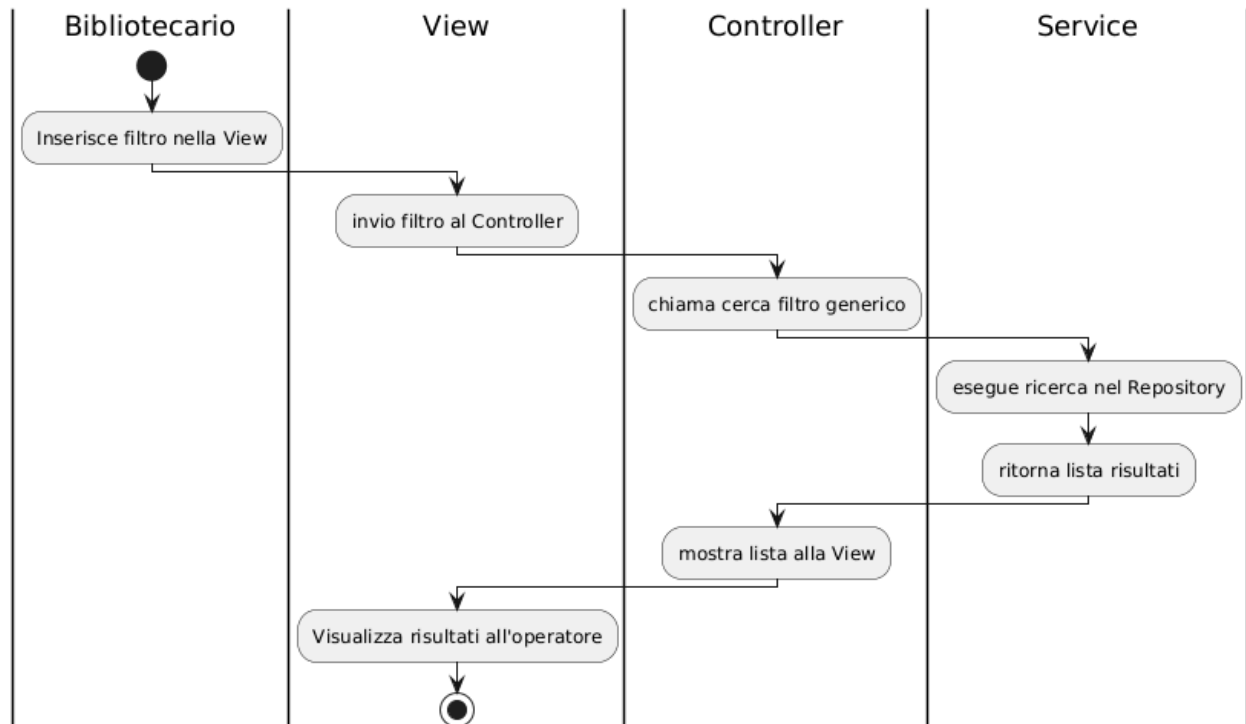
- Il Controller delega a LibroService l'incremento delle copie disponibili del libro.
  - (Tale aggiornamento deve riflettere il fatto che una copia fisica è tornata in magazzino).

## 3. Chiusura del Prestito (PrestitoService)

- PrestitoService acquisisce il controllo e provvede alla chiusura del prestito:
  - Aggiorna la data di restituzione effettiva del prestito (che è stata precedentemente richiesta).
  - Effettua il salvataggio del prestito aggiornato

## 4. Conclusione

1. **Notifica:** Il flusso torna al Bibliotecario per comunicare l'esito di avvenuta restituzione.
2. Il processo si conclude.



## 1. Avvio e Raccolta Dati (Bibliotecario e View)

- **Punto di partenza:** il Bibliotecario inserisce il filtro in base al quale intende effettuare la ricerca nella View, avviando il processo.
- **Delegazione:** La View riceve il filtro inserito e ne effettua l'invio al Controller.

## 2. Orchestrazione (Controller)

- **Logica di Controllo:** Il Controller riceve il filtro passato dalla view, senza tuttavia doverlo impiegare direttamente per effettuare la ricerca.
- **Invocazione del Service:** Il Controller effettua la chiamata alla funzione di ricerca generica sul Service.

## 3. Logica di Business e Accesso Dati (Service)

- **Implementazione della funzionalità di ricerca:** Il Service riceve la richiesta di effettuare la ricerca, occupandosene dell'effettiva implementazione. Tuttavia, l'accesso ai dati viene delegato al Repository.
- **Interazione con i Dati:** Il Repository effettua l'accesso al file, cercando i dati richiesti.
- **Restituzione dei Risultati:** Il Repository termina la ricerca ed il Service ritorna la lista dei risultati al Controller.

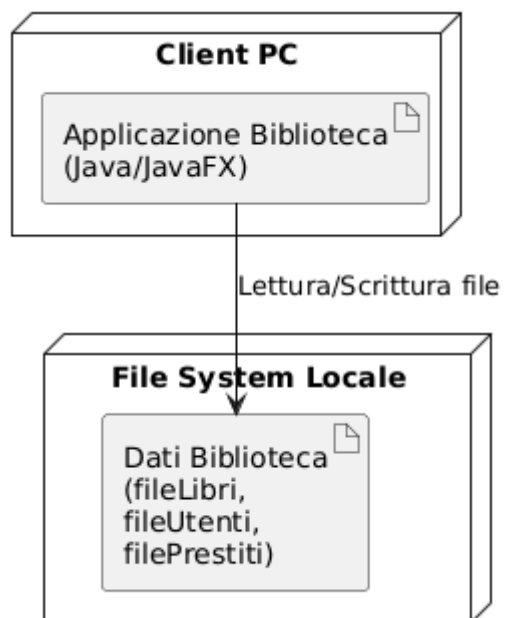
## 4. Presentazione dei Risultati (Controller e View)

- **Aggiornamento della View:** Il Controller riceve la lista dei risultati dal Service e procede ad effettuare l'invio alla View, mediante l'azione **mostra lista alla View**.

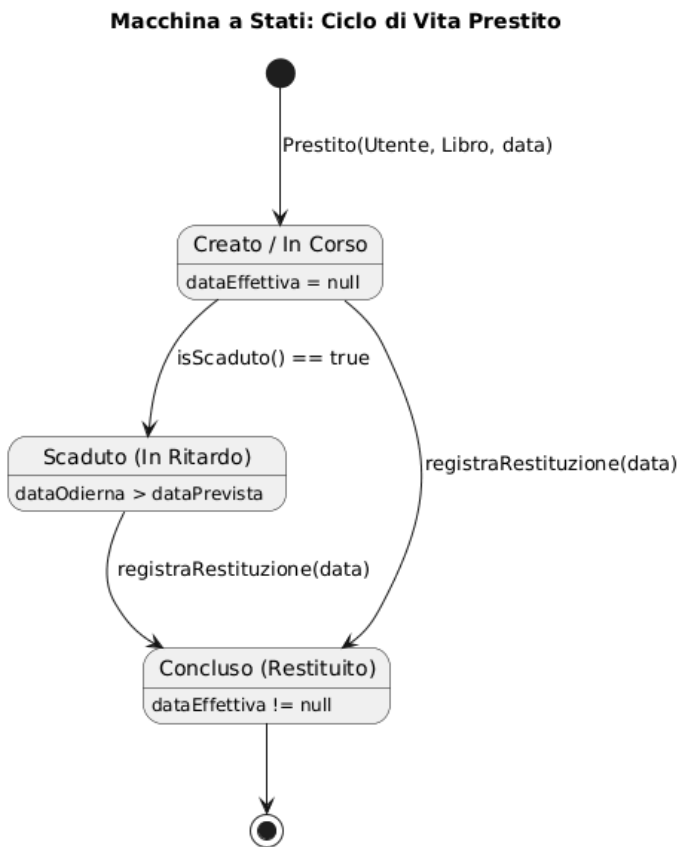
- **Output Finale:** La View gestisce l'attività finale di **Visualizzazione dei risultati**, portando a termine il ciclo e rendendo l'output visibile al Bibliotecario.

## Diagramma di distribuzione

Il Client PC ospita l'applicazione Biblioteca, realizzata in Java. Il salvataggio dei dati avviene in 3 file diversi, memorizzati all'interno del file system locale.



# Diagramma di stati



## Stati e Flussi del Ciclo di Vita Prestito

Il prestito passa attraverso tre stati principali, in base alla data in cui avviene la restituzione.

### 1. Creato / In Corso

Stato iniziale

- **Condizione:** La data di restituzione effettiva (**dataEffettiva**) non è stata ancora inserita, per cui viene inizializzata al valore null. Il prestito è attivo.
- **Ingresso:** Si giunge allo stato Creato all'avvio (**Prestito(Utente, Libro, data)**).
- **Transizioni:**
  - **A Scaduto:** Si passa allo stato **Scaduto** se il sistema verifica che la sia soddisfatta la condizione **isScaduto() == true**, che si verifica quando la data odierna è cronologicamente successiva rispetto alla **dataPrevista**.
  - **A Concluso:** Il passaggio allo stato **Concluso** avviene se la restituzione del libro avviene in tempo, attraverso l'invocazione del metodo **registraRestituzione(data)**.

### 2. Scaduto (In Ritardo)

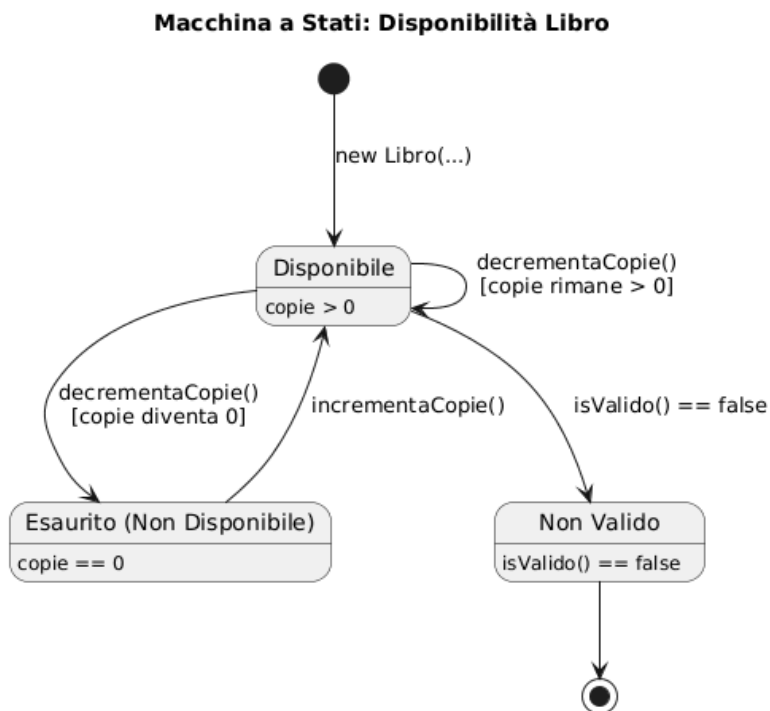
Eventuale stato intermedio del prestito

- **Condizione:** La data odierna è successiva alla **dataPrevista**, e la restituzione del libro non è ancora stata effettuata.
- **Transizione:** Il prestito passa dallo stato **Scaduto** allo stato **Concluso**, mediante l'invocazione del metodo **registraRestituzione(data)**. La restituzione risulta in ritardo.

### 3. Concluso (Restituito)

Stato finale del prestito

- **Condizione:** La data di restituzione effettiva (**dataEffettiva**) è stata impostata, ed il suo valore è diverso da null.
- **Ingresso:** Tale stato è raggiungibile da due stati alternativi: **InCorso**, se la restituzione avviene in tempo, o **Scaduto**, se la restituzione avviene in ritardo.
- **Uscita:** Il prestito è concluso.



## Stati e Flussi di Disponibilità Libro

Il sistema modella la disponibilità di un oggetto Libro, basandosi principalmente sulla quantità di copie a disposizione e sulla validità dei dati inseriti.

### 1. Disponibile

Stato ordinario in cui si trova un oggetto Libro.

- **Condizione:** Sono presenti copie del libro, il che equivale al soddisfacimento della condizione **copie > 0**. In tal caso, il libro è disponibile al prestito.

- **Ingresso:** Si giunge allo stato **Disponibile** all'avvio, mediante la creazione di un nuovo libro, ossia tramite **new Libro(...)**, o quando una copia viene restituita.
- **Azioni:**
  - **Prestito:** l'invocazione del metodo **decrementaCopia()** riporta allo stato **Disponibile** se il numero di copie resta positivo.
  - **Restituzione:** l'invocazione del metodo **incrementaCopia()** dallo stato **Esaurito**, riporta allo stato **Disponibile**.

## 2. Esaurito (Non Disponibile)

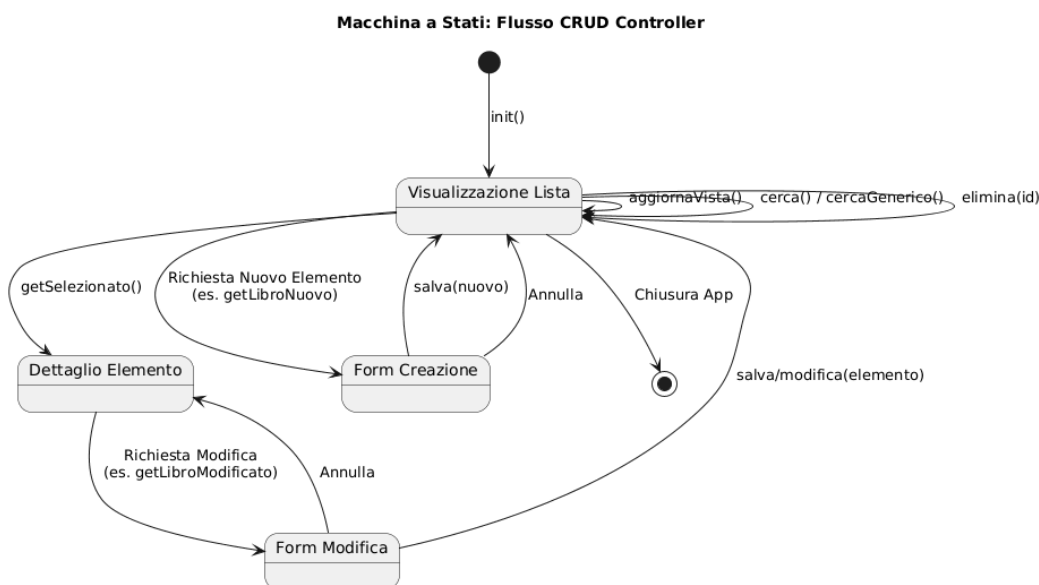
Non sono fisicamente disponibili copie del libro da concedere in prestito.

- **Transizione:** Il libro passa dallo stato **Disponibile** allo stato **Esaurito** nel momento in cui l'invocazione del metodo **decrementaCopia()** è tale da azzerare il numero di copie disponibili.
- **Condizione:** Non sono presenti copie disponibili, il che equivale a soddisfare la condizione **copie == 0**. Non risulta possibile concedere nuovi prestiti.
- **Uscita:**
  - **Ritorno:** Il passaggio dallo stato **Esaurito** allo stato **Disponibile** avviene attraverso la chiamata del metodo **incrementaCopia()**, invocato al momento della restituzione di una copia.

## 3. Non Valido

Tale flusso si occupa di gestire i libri i cui dati risultano non validi

- **Transizione:** Il passaggio dallo stato **Disponibile** allo stato **Non Valido** avviene se risulta soddisfatta la condizione **isValidato() == false**.
- **Uscita:** Un libro non valido viene rimosso dal ciclo operativo.



# Stati e Flussi del Controller CRUD

Il sistema passa attraverso quattro stati principali, guidati dalle azioni intraprese dall'utente.

## 1. Visualizzazione Lista

Stato iniziale dell'applicazione, nonché il principale.

- **Ingresso:** Si giunge allo stato **Visualizzazione Lista** all'avvio, mediante il metodo **init()**. Il controller adopera l'interfaccia **InterfaceView** per mostrare i dati caricati.
- **Azioni:** L'amministratore può eseguire varie operazioni, quali l'aggiornamento della vista tramite il metodo **aggiornaVista()**, la ricerca o l'eliminazione degli elementi, senza portare ad una variazione dello stato corrente.

## 2. Form Creazione

Stato di creazione di un nuovo elemento, che porta al passaggio dalla visualizzazione della lista alla visualizzazione del form di creazione.

- **Transizione:** Il passaggio dallo stato di **Visualizzazione Lista** allo stato di **Form Creazione** avviene mediante una **Richiesta Nuovo Elemento**, che può essere effettuata tramite un metodo get.
- **Uscita:**
  - **Successo:** Passaggio dallo stato di **Form Creazione** allo stato di **Visualizzazione Lista**, tramite **salva(nuovo)**.
  - **Annullamento:** Passaggio dallo stato di **Visualizzazione Lista** allo stato di **Creazione Lista** tramite **Annulla**.

## 3. Form Modifica

Stato in cui è possibile ispezionare gli elementi creati o apportarne modifiche.

- **Dettaglio:** Attraverso il metodo **getSelezionato()**, è possibile passare dallo stato di **Visualizzazione Lista** allo stato di **Dettaglio Elemento**.
- **Modifica:** Il passaggio dallo stato **Dettaglio** allo stato **Form Modifica** avviene mediante una **Richiesta Modifica**, che consiste nell'invocazione di un metodo get.
- **Uscita da Modifica:**
  - **Successo:** L'uscita dallo stato di **Form Modifica** avviene attraverso il salvataggio o la modifica di un elemento, tramite gli omonimi metodi **salva/modifica(elemento)**. Si ritorna allo stato di **Visualizzazione Lista**.
  - **Annullamento:** **Annulla** porta al passaggio dallo stato di **Form Modifica** allo stato **Dettaglio**.

## 4. Fine

La Chiusura App porta dallo stato di **Visualizzazione Lista** alla terminazione.

## Livelli di Coesione

### Model

| Classe/Interfaccia | Livello di Coesione | Giustificazione del livello di coesione attribuito                                      |
|--------------------|---------------------|-----------------------------------------------------------------------------------------|
| InterfaceID        | Funzionale          | L'interfaccia presenta esclusivamente un metodo, pertanto effettua un'unica operazione. |
| Dati               | Funzionale          | La classe astratta raggruppa funzionalità legate alla rappresentazione.                 |
| Libro              | Funzionale          | La totalità dei metodi della classe è finalizzata alla gestione dell'entità Libro.      |
| Utente             | Funzionale          | Tutti i metodi della classe sono adibiti alla gestione dell'entità Utente.              |
| Prestito           | Funzionale          | Tutti i metodi sono preposti alla rappresentazione dell'entità Prestito.                |

### Repository

| Classe/Interfaccia    | Livello di Coesione | Giustificazione del livello di coesione attribuito               |
|-----------------------|---------------------|------------------------------------------------------------------|
| InterfaceGestoreIO<T> | Funzionale          | L'interfaccia mette a disposizione unicamente metodi preposti al |

|                        |                 |                                                                                                     |
|------------------------|-----------------|-----------------------------------------------------------------------------------------------------|
|                        |                 | salvataggio e al caricamento dei dati.                                                              |
| GestoreFile<T>         | Funzionale      | La classe mette a disposizione unicamente metodi preposti al salvataggio e al caricamento dei dati. |
| InterfaceRepository<T> | Comunicazionale | I metodi messi a disposizione dall'interfaccia operano tutti sulla medesima struttura dati.         |
| Repository<T>          | Comunicazionale | Tutti i metodi forniti dalla classe agiscono sulla medesima struttura dati.                         |

## Service

| Classe/Interfaccia  | Livello di Coesione | Giustificazione del livello di coesione attribuito                                                                      |
|---------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| InterfaceService<T> | Funzionale          | L'interfaccia mette a disposizione unicamente metodi che effettuano operazioni sul repository.                          |
| BaseService<T>      | Comunicazionale     | La classe fornisce unicamente metodi che effettuano operazioni sul repository.                                          |
| LibroService        | Funzionale          | La classe mette a disposizione esclusivamente metodi che consentono di implementare la funzione di ricerca di un libro. |
| UtenteService       | Funzionale          | La classe mette a disposizione unicamente un metodo.                                                                    |

|                 |             |                                                                                                       |
|-----------------|-------------|-------------------------------------------------------------------------------------------------------|
| PrestitoService | Procedurale | La classe mette a disposizione metodi che implementano funzionalità adoperate insieme frequentemente. |
|-----------------|-------------|-------------------------------------------------------------------------------------------------------|

## View

| Classe/Interfaccia | Livello di Coesione | Giustificazione del livello di coesione attribuito                                                                                   |
|--------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| InterfaceView<T>   | Funzionale          | Tutti i metodi messi a disposizione dall'interfaccia implementano funzionalità legate alla visualizzazione.                          |
| BaseView<T>        | Comunicazionale     | Tutti i metodi messi a disposizione dalla classe implementano funzionalità legate alla visualizzazione.                              |
| LibroView          | Funzionale          | I metodi messi a disposizione della classe permettono di visualizzare i dati di un libro all'atto della creazione o della modifica.  |
| UtenteView         | Funzionale          | I metodi messi a disposizione della classe permettono di visualizzare i dati di un utente all'atto della creazione o della modifica. |
| PrestitoView       | Funzionale          | I metodi forniti dalla classe hanno il compito di visualizzare i dati inseriti all'atto della creazione di un nuovo prestito.        |

## Controller

| Classe/Interfaccia  | Livello di Coesione | Giustificazione del livello di coesione attribuito                                                                        |
|---------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------|
| InterfaceController | Funzionale          | I metodi messi a disposizione dall'interfaccia hanno lo scopo di inizializzare e aggiornare la visualizzazione.           |
| BaseController<T>   | Comunicazionale     | La classe mette a disposizione metodi adibiti al coordinamento della view e del service.                                  |
| CrudController<T>   | Logica              | La classe fornisce metodi che si occupano di effettuare operazioni su un'entità generica.                                 |
| LibroController     | Funzionale          | La totalità dei metodi messi a disposizione dalla classe implementa funzionalità legate alla gestione dell'entità Libro.  |
| UtenteController    | Funzionale          | La totalità dei metodi messi a disposizione dalla classe implementa funzionalità legate alla gestione dell'entità Utente. |
| PrestitoController  | Funzionale          | I metodi forniti dalla classe implementano funzionalità legate ai prestiti.                                               |

## Livelli di accoppiamento:

### Accoppiamento per Dati

Lo scambio di informazioni tra moduli coinvolge solo informazioni strettamente necessarie al funzionamento del sistema. Rappresenta il massimo livello di accoppiamento possibile.

## MODEL:

- **Libro:**

**Libro(String titolo, String isbn, List<String> autori, int anno, int copie):** Il metodo acquisisce in input le sole informazioni necessarie alla definizione di un'istanza della classe Libro.

## REPOSITORY:

- **InterfaceGestoreIO<T> e GestoreFile<T>:**

- **void salvaDati(String nomeFile, List<T> dati):** Il metodo prende come parametro di ingresso unicamente il nome del file su cui effettuare il salvataggio e i dati da salvare.
- **List<T> caricaDati(String nomeFile):** Il metodo prende come unico parametro di ingresso il nome del file su cui sono salvati i dati.

- **InterfaceRepository<T> e Repository<T>:**

- **void elimina(String id):** Il metodo richiede unicamente un parametro di input, rappresentato dall'identificativo univoco dell'elemento di cui occorre effettuare l'eliminazione.
- **T cerca(String id):** L'identificativo univoco dell'elemento da cercare costituisce l'unico parametro necessario al metodo per effettuare la ricerca dell'elemento desiderato.

## SERVICE:

- **InterfaceService<T> e BaseService<T>:**

- **T cerca(String id):** Il metodo acquisisce in input il solo id dell'elemento di cui si deve effettuare la ricerca.
- **List<T> cercaGenerico(String filtro):** Il parametro di input del metodo è rappresentato unicamente dalla stringa adoperata per filtrare opportunamente i risultati di ricerca.

- **LibroService:**

- **List<Libro> cercaPerTitolo(String titolo):** Il metodo prende in ingresso unicamente il titolo del libro, ai fini della ricerca.

- **UtenteService:**

- **UtenteService: List<Utente> cercaPerCognome(String cognome):** Il metodo ha un solo parametro di input, rappresentato dal cognome dell'utente di cui effettuare la ricerca.
- **PrestitoService:**
  - **void registraRestituzione(Prestito prestito, LocalDate dataEffettiva):** I parametri di input del metodo sono rappresentati dalle sole informazioni necessarie alla registrazione del prestito, quali il prestito effettuato (che include tutti i dati relativi all'utente richiedente, al libro concesso e alla data di terminazione del prestito), e la data di effettiva restituzione del libro.

## VIEW:

- **InterfaceView<T> e BaseView<T>:**
  - **void mostraMessaggio(String messaggio):** Il metodo acquisisce in input il solo testo da visualizzare.

## CONTROLLER:

- **BaseController<T>:**
  - **void eseguiOperazione(Runnable operazione, String messaggioConferma):** Il metodo prende esclusivamente in ingresso l'operazione specifica da effettuare e la stringa che segnala l'esito.
- **CrudController<T>:**
  - **void elimina():** Il metodo effettua la chiamata.

## Accoppiamento per timbro (Stamp coupling)

Lo scambio di informazioni tra moduli coinvolge anche informazioni che non sono strettamente necessarie. Rappresenta il secondo livello di accoppiamento più alto possibile.

## REPOSITORY:

- **InterfaceRepository<T> e Repository<T>:**
  - **void inserisciOAggiorna(T elemento):** La funzionalità di aggiornamento, implementata dal metodo, necessita del passaggio di un'istanza della classe T, sebbene possa esser sufficiente aggiornare solo una parte dei valori degli attributi dell'oggetto.

## SERVICE:

- **InterfaceService<T> e BaseService<T>:**
  - **void salva(T elemento):** Il metodo riceve in ingresso un'istanza della classe T. Tuttavia, il salvataggio potrebbe non essere necessario per la totalità degli attributi dell'oggetto.
  - **void modifica(T elemento):** Il metodo riceve in ingresso un oggetto di tipo elemento, provvisto di tutti gli attributi e di tutti i metodi della classe di appartenenza. Tuttavia, la modifica può non interessare tutti gli attributi dell'oggetto passato in input.

## VIEW:

- **LibroView:**
  - **Libro getLibroModificato():** Il metodo visualizza l'intero libro in seguito alla procedura di modifica, includendo anche i dati sono rimasti invariati.

## Matrice di tracciabilità

| Modulo | Tipo di Requisito | ID Requisito           |
|--------|-------------------|------------------------|
| MODEL  | Dati (DF)         | DF-1.0, DF-2.0, DF-3.0 |
|        | Validazione (IF)  | IF-1.0.2, IF-2.0.2     |

|            |                      |                                              |
|------------|----------------------|----------------------------------------------|
| REPOSITORY | Persistenza (IS)     | IS-1.0.3, IS-2.0.3, IS-3.0.3, IS-3.1.3       |
| SERVICE    | Ricerca (IF)         | IF-1.2.1, IF-1.2.2, IF-2.2.1, IF-2.2.2       |
|            | Vincoli (FC)         | FC-3.0.1, FC-3.0.2                           |
|            | Ordinamento (FC)     | FC-1.2.1, FC-2.2.1, FC-3.1.1                 |
| VIEW       | Input (IF)           | IF-1.0.1, IF-2.0.1                           |
|            | Output/Messaggi (UI) | UI-1.0.2, UI-2.0.2.1, UI-3.0.1.1, UI-3.0.2.1 |
| CONTROLLER | Flusso (BF)          | BF-1.0, BF-1.1, BF-2.0, BF-3.0, BF-3.1       |
|            | Azioni (IF)          | IF-1.1.1, IF-1.1.2, IF-2.1.1, IF-2.1.2       |